

1. prompt

- 已了解。

- $1/5$

任务)。

GPT-5.3-Codex-Medium:

模块 1 已完成并落地到 Streamlit 应用中，核心代码在 app.py，依赖清单在 requirements.txt。

已实现内容对照你的 4 条要求：

1. 数据获取与解析

- 使用 requests 从 NeuralEDUSeg 的原始数据地址抓取同名样本对：.out 与 .edus。
- 当前内置可选样本如 wsj_0601.out、wsj_0604.out、wsj_0615.out。
- 解析 .edus 逐行得到 Ground Truth EDU 切分。

2. 规则基线 (spaCy)

- 加载 en_core_web_sm (本地无模型时自动下载)。
- 对同一 .out 纯文本做依存分析，并用启发式规则切分：
 - 强标点切分
 - SCONJ 从属连词切分
 - 部分从句相关依存模式切分
- 输出规则基线 EDU 序列。

3. 可视化对比视图

- 第 1 标签页采用左右两栏：
 - 左栏：规则基线切分结果
 - 右栏：NeuralEDUSeg 真实标注结果

4. EDU 卡片 + 边界词高亮

- 每个 EDU 以独立边框卡片显示。
- 在词级别高亮边界词 (每个 EDU 的边界 token)。
- 页面下方额外给出左右两侧边界词列表，便于观察序列标注效果。

另外已按你的背景要求保留 3 模块结构：

- 模块 1 已实现
- 模块 2、模块 3 为占位页，便于你继续分模块迭代

说明：

- 我做了静态检查，当前唯一问题是环境里还未安装 streamlit 和 spacy (属于缺依赖，不是代码语法错误)。

你现在可以直接运行：

1. 安装依赖：pip install -r requirements.txt
2. 启动应用：streamlit run app.py

- 模块 2：浅层篇章分析与显式关系提取
 1. 对应内容：PDTB 与显式篇章关系、连接词消歧
 2. 设计意图：验证基于 PDTB 框架的浅层篇章分析，重点提取“显式连接词”并划分 Arg1 和 Arg2。
 3. 具体要求：在第二个标签页中，实现浅层篇章分析（基于显式连接词）。
 - 提供一个句子输入框，默认输入课件中的例子：'Third-quarter sales in Europe were exceptionally strong, boosted by promotional programs and new products - although weaker foreign currencies reduced the company's earnings.'
 - 规则匹配：在后台定义一个常见的显式连接词列表及其所属的 PDTB 顶级分类（如 Temporal: when/after; Contingency: because/since; Comparison: but/although; Expansion: and/or）。
 - 扫描输入句子，如果发现这些连接词，在页面上将其加粗并用醒目的颜色标注其语义类别（如：although [COMPARISON]）。
 - 论据提取（简易版）：以该连接词为界，结合刚才模块 1 的思想，尝试将句子切分为前置论据（Arg1）和后置论据（Arg2），并分别用不同颜色的背景框显示。

GPT-5.3-Codex-Medium:

模块 2 已完成，并已接入第二个标签页。

本次实现内容：

1. 句子输入框
 - 在模块 2 添加了默认示例句，与你给定文本一致。
2. 规则匹配（连接词 + PDTB 顶级分类）
 - 新增常见显式连接词词表，并映射到四类关系：TEMPORAL、CONTINGENCY、COMPARISON、EXPANSION。
 - 对输入句子进行扫描，支持大小写不敏感匹配。
3. 连接词高亮展示
 - 在页面中将命中的连接词加粗，并附带类别标签展示，例如 although [COMPARISON]。
 - 使用醒目颜色区分不同关系类型。
4. Arg1 / Arg2 简易提取
 - 以句中第一个命中的显式连接词作为切分点。
 - 连接词前文本作为 Arg1，后文本作为 Arg2。
 - 两个论据用不同底色卡片并排展示，便于对比观察。

说明：

- 当前静态检查仍提示缺少本地依赖 (streamlit、spacy)，属于环境问题，不是本次逻辑错误。
- 你在 py310 环境下可直接运行安装与启动：

1. pip install -r requirements.txt
2. streamlit run app.py

- 模块 3：指代消解 (Coreference Resolution) 可视化
 1. 对应内容：指代消解概念、表述发现、排序与神经网络端到端消解
 2. 设计意图：将文字示例转化为程序可自动识别并高亮显示的聚类结果。
 3. 具体要求：在第三个标签页中，实现指代消解功能。
 - 提供一个多行文本输入框，默认文本为一段包含多个人称代词的英文段落（例如课件 P64 的英文版本或一段维基百科简介）。
 - 后台使用 fastcoref 库对输入文本进行分析（它类似于课件 P75 提到的端到端 E2E-COREF 神经网络模型）。
 - 核心功能：提取文本中的所有指代簇 (Coreference Clusters)。将属于同一个实体 (Entity) 的不同表述 (Mention) 用同一种背景颜色在原始段落中 HTML 高亮渲染出来。
 - 在高亮文本下方，用结构化的列表输出每个“等价类”，例如：Cluster 1: ['Barack Obama', 'He', 'his']。

GPT-5.3-Codex-Medium：

模块 3 已完成，并已接入第三个标签页。

本次实现

1. 在第三标签页新增多行文本输入框，内置一段含代词的英文默认段落。
2. 后台接入 fastcoref，执行指代消解并提取 Coreference Clusters。
3. 将同一实体簇中的 Mention 在原文中用同一种背景色高亮显示（带 Cluster 提示）。
4. 在高亮文本下方按结构化列表输出等价类，如 Cluster 1: [...], Cluster 2: [...]
5. 增加健壮性处理：缺少 fastcoref 时会在页面提示安装命令；运行异常会给出报错信息。

当前检查结果

- 代码结构无新增语法问题。
- 目前编辑器仍提示无法解析 spacy、streamlit、fastcoref 导入，这是本地环境尚未安装依赖导致。

你现在可以在 py310 环境执行

1. pip install -r requirements.txt
2. streamlit run app.py

2. 实验总结

本次实验围绕“篇章分析综合平台”的搭建，较完整地把课堂中的核心概念转化为可运行、可观察、可比较的流程。模块 1 通过规则切分与 NeuralEDUSeg 真实标注的并排对照，让我直观看到传统启发式方法在可解释性上的优势，也清楚暴露了其在长句、嵌套结构和隐性边界上的局限。模块 2 聚焦 PDTB 显式关系提取，在连接词识别与 Arg1/Arg2 切分实践中，我进一步体会到连接词消歧并非词典匹配即可完成，而是必须结合时态、句法和上下文语义进行判别。模块 3 引入 fastcoref 后，指代簇可视化明显提升了篇章层面的可读性，使同一实体的多种表述从抽象理论变成了可验证的分析结果，同时也让我认识到复杂代词在跨句回指中的不稳定性。整体而言，本次实验实现了从句内分析到篇章理解的层级提升，也加深了我对规则方法与神经方法互补关系的理解：前者便于控制与解释，后者更擅长建模复杂语境。后续若结合更大语料与误差分析机制，系统在鲁棒性和学术价值上还可以进一步提升。

2.1 模块 1 观察任务：观察你编写的简单规则与 NeuralEDUSeg 真实数据集中的人工/模型切分结果有何差异。思考课件 P36 提到的“受限自注意力机制 (Restricted Self-attention)”是如何帮助神经网络在长句子中更准确地捕捉边界的？

- 从对比结果看，规则切分主要依赖标点和连词，优点是直观、稳定，但容易把结构复杂的从句切得过碎，或漏掉无明显触发词的边界。NeuralEDUSeg 的结果更贴近真实语义单元，尤其在长句中能兼顾前后语境。其受限自注意力机制通过聚焦局部关键上下文、抑制远距离噪声干扰，使模型在主从结构、插入语和并列成分中更准确地判断 EDU 边界。

2.2 模块 2 观察任务：输入课件 P51 的两个包含 "since" 的句子。观察你的程序是否能区分 "since" 在不同语境下的语义类别 (TEMPORAL 时间关系 vs CONTINGENCY 因果关系)。体会课件中提到的“显式连接词消歧” (P51) 在实际工程中的难点。

- 在测试两个含 since 的句子后可以看到，当前规则法通常把 since 固定归到同一类别，难以稳定区分“自从……”的时间义和“因为……”的因果义。真正影响判断的是时态搭配、事件先后和上下文语气，这些信息仅靠词表很难覆盖。也正因如此，显式连接词消歧在工程里并不只是匹配触发词，而是一个依赖语境建模的细粒度判别问题。

2.3 模块 3 观察任务：输入一段包含复杂代词 (如 he, she, it, they) 的文本。观察模型是否能正确处理跨句子的回指 (Anaphora)。思考课件 P72 中提到的“基于表述排序 (Mention Ranking)”的算法，在底层是如何给这些代词计算关联分数的？

- 从实验结果看，模型对相邻句中的 he、she 通常能较准确回指，但遇到 it、they 这类语义更宽的代词时，跨句链接更容易受干扰，尤其在候选实体较多时会出现偏差。表述排序的核心是给“当前代词-候选先行词”逐一打分，综合语法一致性、语义相容性、距离与上下文线索，最后选择得分最高的配对并形成实体簇。